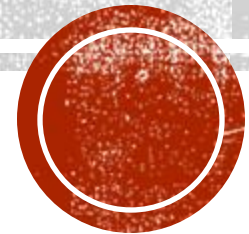


# CLOUD COMPUTING

Chandara Chea

[chandara.chea@cadt.edu.kh](mailto:chandara.chea@cadt.edu.kh)



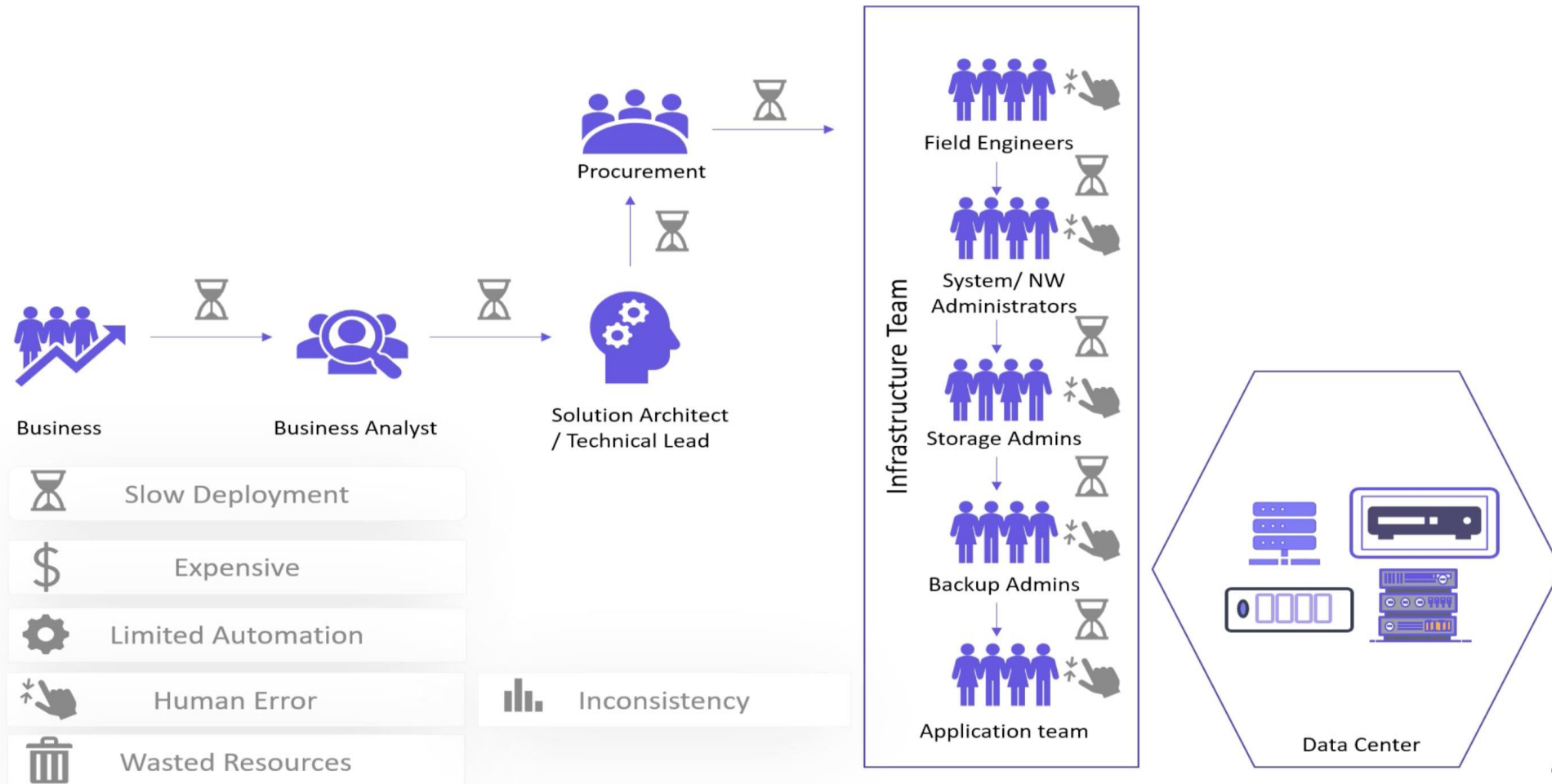
# WEEK 6 — INFRASTRUCTURE AS CODE

- **Learning Objectives:**

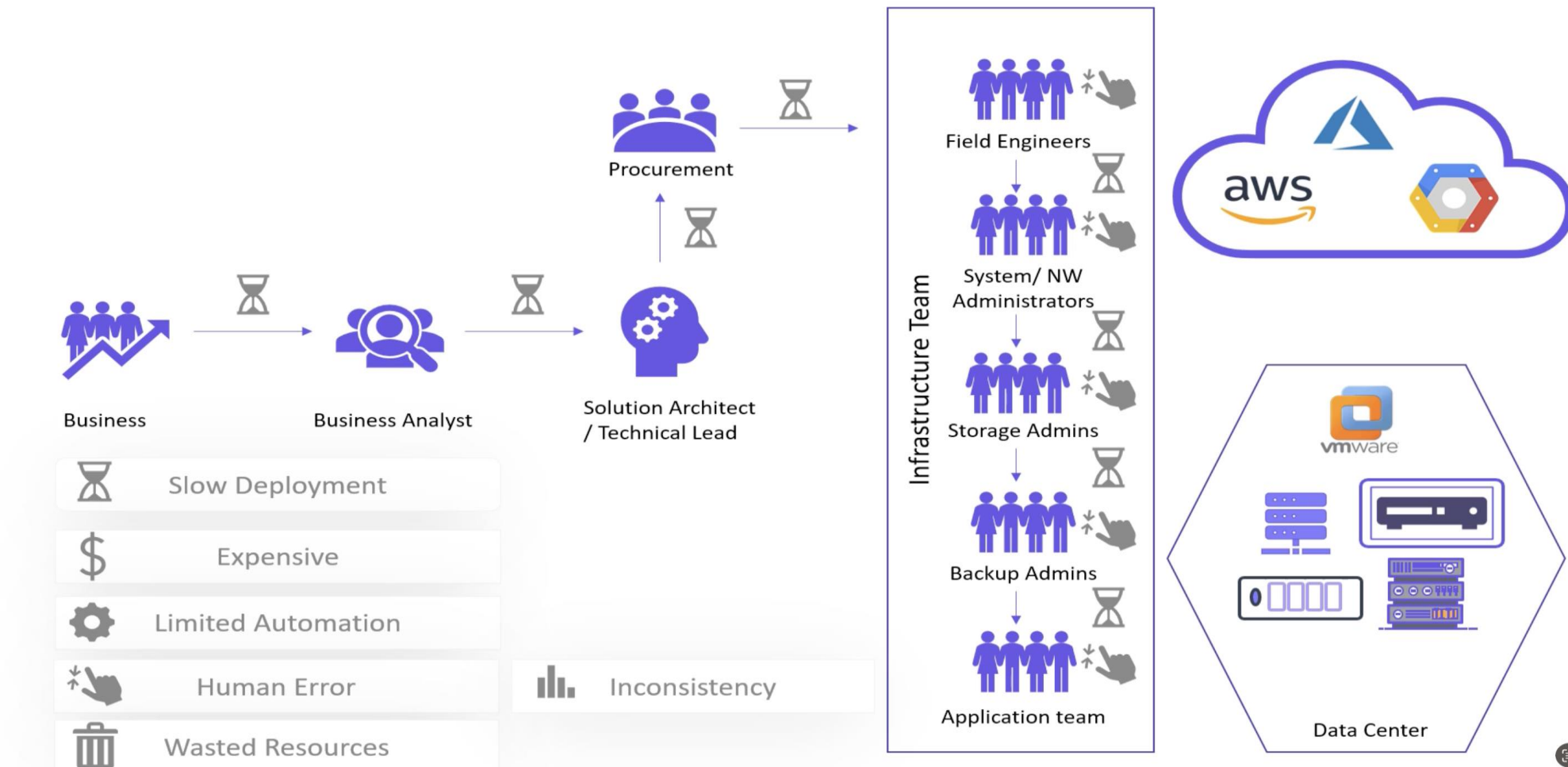
- Understand what is IaC
- Understand what are benefits and best practices of using IaC
- Understand what is Terraform
- Understand Terraform's components



# CHALLENGES WITH TRADITIONAL IT INFRASTRUCTURE



# CHALLENGES WITH TRADITIONAL IT INFRASTRUCTURE



# WHAT IS IAC?

- Process of managing and provisioning infrastructure through machine-readable definitions, rather than manual configuration. IaC allows you to build, change, and manage your infrastructure in a safe, consistent, and repeatable way by defining resource configurations that you can version, reuse, and share.
- Approaches:
  - Imperative (how): define step by step how infrastructure is provision
    - More complexity as it's required a lot of custom script for example to tear down environment or scale up/down
    - E.g. CLI create VPC
  - Declarative (what): define final step of the infrastructure and let the provider to handle the rest
    - More easily manage the configuration
    - E.g. Terraform or AWS cloudformation



# PROBLEMS WITH MANUAL OPERATION

- Reproducibility: difficult to keep track of what changes we had made where.
- Documentation (instruction manual): rare for documents to be kept up to date as changes and improvements are made to the way things are done. And, documents still often leave gaps.
- Variation (differences in versions and configurations): software and scripts that work on some machines don't work on others.



# BENEFITS OF AUTOMATION WITH IAC

- Consistency: any action performed by a human hundreds of times won't be performed the same way each time: very few of us will ever be as consistent as a machine ⇒ Consistency is the primary value of automation.
- Time Saving
- Reducing the effort and risk of making changes to infrastructure
- Improving the speed to troubleshoot and resolve failures
- Creating systems that are reliable, secure, and cost-effective
- Make governance, security, and compliance controls visible



# BEST PRACTICES

- **Keep Infrastructure Code in Version Control**

- Just like application code, infrastructure code should be versioned and stored in a Git repository. This ensures that changes can be tracked and reviewed, and you can roll back to a previous version if something goes wrong.

- **Use Modular and Reusable Components**

- Split your infrastructure code into smaller, reusable modules or components. This approach promotes DRY (Don't Repeat Yourself) principles and makes it easier to manage complex infrastructure.

- **Adopt a "Test First" Approach**

- IaC code should be tested before being applied to production environments. Use tools like **terraform plan** to preview changes, and integrate IaC tests into CI/CD pipelines to ensure infrastructure changes are safe and reliable.

- **Implement Change Management and Review Processes**

- Infrastructure changes should follow the same processes as application code changes. This includes peer reviews, automated tests, and approval workflows before changes are deployed.

- **Keep Infrastructure Code Simple and Readable**

- IaC code should be simple to understand and well-documented. This reduces the risk of errors and makes it easier for other team members to contribute and understand the infrastructure setup.





# INFRASTRUCTURE CHANGES

- Infrastructure management approaches can be broadly categorized into two models: **mutable** and **immutable**. These concepts define how changes to the infrastructure are handled over time.
- **Mutable Infrastructure**: Infrastructure is updated and changed directly, typically through manual configurations or updates.
  - If you need to apply configuration changes, install updates, or fix issues, you modify the existing infrastructure directly.
- **Immutable Infrastructure**: Infrastructure components are replaced rather than modified, ensuring that every change is treated as a new deployment.
  - When an update is needed, a new version of the infrastructure (such as a new virtual machine or container) is provisioned, and the old version is decommissioned.

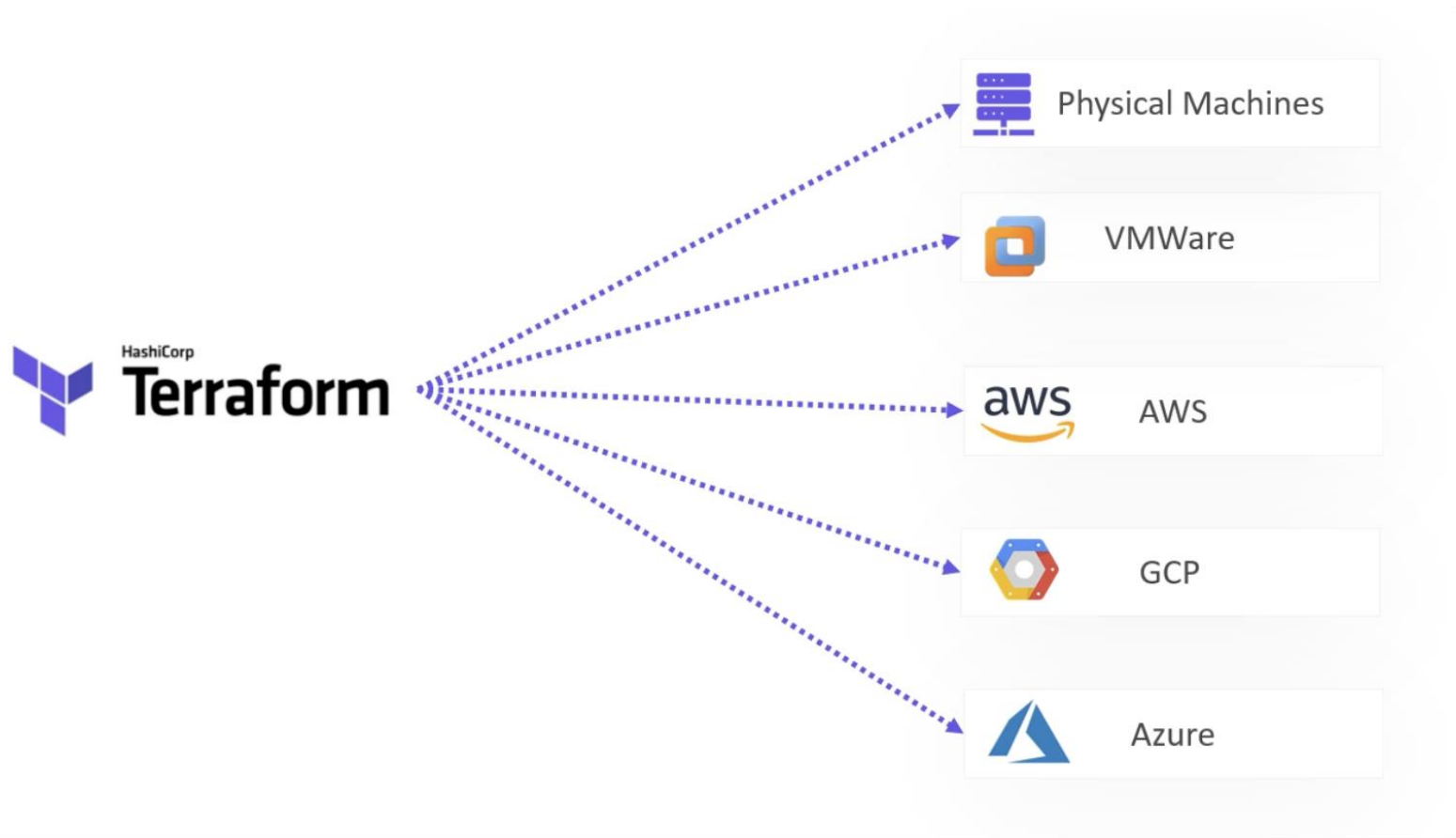


# WHICH APPROACH SHOULD YOU USE?

- **Mutable Infrastructure** is more suited for environments where:
  - There's a need to update existing systems or applications frequently.
  - Teams are accustomed to traditional IT management methods.
  - Systems are smaller, and manual intervention is still manageable.
- **Immutable Infrastructure** is preferred for:
  - Cloud-native or containerized environments (e.g., Docker, Kubernetes).
  - High-frequency deployments where consistency and fast rollbacks are crucial.
  - Applications where uptime, reliability, and security are top priorities, especially in distributed systems.
  - Teams that can invest in automation and have a robust CI/CD pipeline in place.



# WHAT IS TERRAFORM?



# WHAT IS TERRAFORM?

- Terraform is HashiCorp's infrastructure as code tool. It lets you define resources and infrastructure in human-readable, declarative configuration files, and manages your infrastructure's lifecycle. Using Terraform has several advantages over manually managing your infrastructure:
  - Terraform can manage infrastructure on multiple cloud platforms.
  - The human-readable configuration language helps you write infrastructure code quickly.
  - Terraform's state allows you to track resource changes throughout your deployments.
  - You can commit your configurations to version control to safely collaborate on infrastructure.
- [https://www.youtube.com/watch?v=tomUWcQ0P3k&ab\\_channel=Fireship](https://www.youtube.com/watch?v=tomUWcQ0P3k&ab_channel=Fireship)



# CONFIGURATION LANGUAGE

```
main.tf

resource "aws_instance" "webserver" {
  ami          = "ami-0edab43b6fa892279"
  instance_type = "t2.micro"
}

resource "aws_s3_bucket" "finance" {
  bucket = "finanace-21092020"
  tags = {
    Description = "Finance and Payroll"
  }
}

resource "aws_iam_user" "admin-user" {
  name = "lucy"
  tags = {
    Description = "Team Leader"
  }
}
```



# CONFIGURATION LANGUAGE

main.tf

```
resource "aws_instance" "webserver" {  
  ami          = "ami-0edab43b6fa892279"  
  instance_type = "t2.micro"  
}  
  
resource "aws_s3_bucket" "finance" {  
  bucket = "finanace-21092020"  
  tags   = {  
    Description = "Finance and Payroll"  
  }  
}  
  
resource "aws_iam_user" "admin-user" {  
  name = "lucy"  
  tags = {  
    Description = "Team Leader"  
  }  
}
```

Init

Plan

Apply

## Real World Infrastructure



# TERRAFORM CONFIGURATION

- The core Terraform workflow consists of three main steps after you have written your Terraform configuration:
  - **Initialize** prepares your workspace so Terraform can apply your configuration.
  - **Plan** allows you to preview the changes Terraform will make before you apply them.
  - **Apply** makes the changes defined by your plan to create, update, or destroy resources.



# PROVIDERS

- Providers are the plugins that allow Terraform to work with different platforms like AWS, Azure, GCP, Kubernetes, etc.
- Each provider manages a set of resources.
- You must define which provider to use in your code.

```
resource "aws_instance" "web" {  
    ami      = "ami-12345678"  
    instance_type = "t2.micro"  
}
```





# RESOURCES

- Terraform uses resource blocks to manage infrastructure, such as virtual networks, compute instances, or higher-level components such as DNS records. Resource blocks represent one or more infrastructure objects in your Terraform configuration.
- Most Terraform providers have a number of different resources that map to the appropriate APIs to manage that particular infrastructure type.

| Generic    | AWS Provider<br>Resource | AWS<br>Infrastructure |
|------------|--------------------------|-----------------------|
| Resource A | aws_instance             | EC2 Instance          |
| Resource B | aws_security_group       | Security Group        |



# VARIABLES

- Variables make your code flexible and reusable.
- You can pass values during execution or define defaults.

```
variable "region" {  
    default = "us-east-1"  
}
```



# MODULES

- Modules are reusable Terraform configurations (like functions in programming).
- You can define a module once and use it multiple times.

```
module "vpc" {  
    source = "./vpc"  
    cidr_block = "10.0.0.0/16"  
}
```



# STATE FILE (TERRAFORM.TFSTATE)

- Terraform tracks the current state of your infrastructure in a file.
- It compares the current state with your code and applies only the differences.
- Keep it secure! The state file may contain sensitive data like passwords or API keys.



# HOMEWORK

- Follow instruction to install terraform  
<https://developer.hashicorp.com/terraform/tutorials/aws-get-started/install-cli>
- Submission: screenshot your whole screen of running nginx in your web browser

